

UR5e + ROS 2 Humble: Hello World Tutorial (Updated for Docker Subnet Conflict)

This updated tutorial explains how to configure networking on a dedicated subnet to avoid conflicts with Docker bridges, then launch the UR ROS 2 driver and send a first 'Hello World' motion command to the UR5e robot. It also includes troubleshooting steps and solutions for typical connectivity and control issues.

1. Network Setup (Avoiding Docker Conflicts)

Goal: Connect the PC (ROS 2 machine) to the UR5e controller without disabling Wi-Fi, and without conflicting with Docker or VirtualBox networks that often use 192.168.56.0/24.

Because Docker automatically creates a bridge network on 192.168.56.0/24, we move the robot network to a clean subnet 192.168.57.0/24. This keeps Docker intact and isolates the robot communication.

```
# Configure Ethernet with NetworkManager (PC side)
nmcli con add type ethernet ifname enp4s0 con-name UR \
  ipv4.method manual ipv4.addresses 192.168.57.1/24 \
  ipv4.never-default yes ipv6.method ignore autoconnect yes

# Give Wi-Fi higher priority for internet
nmcli con mod UR ipv4.route-metric 2000
nmcli con mod Tec ipv4.route-metric 100
nmcli con up UR
```

Verify routes and ping the robot:

```
ip route
ping -I enp4s0 -c 4 192.168.57.101
```

2. Robot-Side Setup (Teach Pendant)

1. On the pendant, create a program with URCaps → External Control. 2. Set IP = 192.168.57.1 (PC), Port = 50002. 3. Save as external_control.urp. 4. Switch robot to Remote Control, Power On, Release Brakes.

3. Run the ROS 2 Driver

```
source /opt/ros/humble/setup.bash
source ~/workspace/ros_ur_driver/install/setup.bash

ros2 launch ur_robot_driver ur_control.launch.py \
  ur_type:=ur5e robot_ip:=192.168.57.101 launch_rviz:=false
```

4. Start the Program from ROS (Dashboard Client)

```
ros2 service list | grep dashboard_client

ros2 service call /dashboard_client/power_on std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/brake_release std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/play std_srvs/srv/Trigger "{}"
```

5. Controllers

```
ros2 service call /controller_manager/list_controllers \
  controller_manager_msgs/srv/ListControllers "{}"

ros2 service call /controller_manager/switch_controller \
  controller_manager_msgs/srv/SwitchController "{
  start_controllers: ['scaled_joint_trajectory_controller'],
```

```

    stop_controllers: [],
    strictness: 2,
    activate_asap: true,
    timeout: 5.0
}"

```

6. First Motion Command (Hello World)

Read current joint states:

```
ros2 topic echo --once /joint_states
```

Example motion (nudging J1 by +0.05 rad):

```

ros2 action send_goal /scaled_joint_trajectory_controller/follow_joint_trajectory \
control_msgs/action/FollowJointTrajectory "{
  trajectory: {
    joint_names: [
      'shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint',
      'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint'
    ],
    points: [{
      positions: [-0.0412247, -1.5992855, -1.4604226,
                  -1.4777158, -1.6276873, 1.5959989],
      time_from_start: {sec: 2}
    }]
  }
}"

```

7. Common Problems & Fixes

Problem 1: Wi-Fi dropped when configuring Ethernet Cause: Ethernet route took over as default. Fix: Configure Ethernet with never-default and set route metrics so Wi-Fi stays preferred.

```

nmcli con mod UR ipv4.never-default yes
nmcli con mod UR ipv4.route-metric 2000
nmcli con mod Tec ipv4.route-metric 100
nmcli con up UR

```

Problem 2: Play button greyed out in Remote Cause: In Remote Control mode, Polyscope disables the physical Play button. Fix: Start the program via dashboard services:

```

ros2 service call /dashboard_client/power_on std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/brake_release std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/play std_srvs/srv/Trigger "{}"

```

Problem 3: Dashboard services stuck on 'waiting for service' Cause: Wrong namespace or driver not running. Fix: List all services and call with the exact namespace:

```

ros2 service list | grep dashboard_client
ros2 service call /ur5e/dashboard_client/play std_srvs/srv/Trigger "{}"

```

Problem 4: Robot did not move even though commands were published Cause: Command controller was inactive. Fix: Activate the desired controller:

```

ros2 service call /controller_manager/list_controllers \
controller_manager_msgs/srv/ListControllers "{}"

ros2 service call /controller_manager/switch_controller \
controller_manager_msgs/srv/SwitchController "{
  start_controllers: ['scaled_joint_trajectory_controller'],
  stop_controllers: [],
  strictness: 2,
  activate_asap: true,
  timeout: 5.0
}"

```

Problem 5: Calibration mismatch warning Cause: URDF kinematics calibration file didn't match the robot's internal calibration. Fix: Extract calibration with ur_calibration tool and update the driver configuration:

```
ros2 run ur_calibration calibration_correction \  
  --robot_ip 192.168.57.101 \  
  --output_path my_robot_calibration.yaml
```

Problem 6: Destination Host Unreachable when pinging robot Cause: Docker created a bridge network on 192.168.56.0/24, conflicting with the robot subnet. Symptom: ping shows 'Destination Host Unreachable' even when robot is on. Fix: Move the robot link to a clean subnet (e.g., 192.168.57.0/24):

```
# PC side (NetworkManager)  
nmcli con mod UR ipv4.addresses 192.168.57.1/24  
nmcli con up UR
```

```
# Robot side (teach pendant → Network)  
IP: 192.168.57.101  
Netmask: 255.255.255.0  
Gateway: 192.168.57.1
```

This avoids IP conflicts with Docker and keeps both networks fully functional.